Studify

Algoritmus Leíró Dokumentáció

Készítette: Fülöp Miklós János & Vadkerti-Tóth Ádám | 13.S1

Tartalomjegyzék

Tartalomjegyzék 1
1. Bevezetés 2
1.1 A dokumentáció célja 2
1.2 A rendszer célja és funkciói 2
1.3 Architektúra áttekintése 2
2. Felhasználói Felület 3
2.1 Technológiai alap 3
2.2 Felépítés és komponenslogika 3
2.3 UI komponensrendszer 3
2.4 Backenddel való kommunikáció 4
3. Backend 5
3.1 Controllerek 5
3.2 Servicek 5
3.3 Repository 5
3.4 Adatbázis 5
3.5 Websocket Gateway 5
4. Regisztráció folyamata (Példa az algoritmusról) 7
4.1 Felület (Frontend) 7
4.2 API és Controller réteg 7
4.3 Service réteg 7
4.4 Repository réteg 8
4.5 Adatbázis réteg (mentési folyamat) 8
4.6 Websocket Gateway 9
4.7 Folyamatábra 10

1. Bevezetés

1.1 A dokumentáció célja

Ez a dokumentáció az Algoritmus rendszer adatmodelljét mutatja be, részletesen ismertetve az entitásokat, azok attribútumait, valamint a köztük fennálló kapcsolati struktúrákat. Célja, hogy átlátható módon bemutassa az algoritmus működését támogató adatszerkezeteket, ezzel elősegítve a fejlesztési, optimalizálási, tesztelési és karbantartási feladatokat.

A dokumentáció referenciaként szolgál a rendszer továbbfejlesztése, teljesítményhangolása és más rendszerekkel való integrációja során.

1.2 A rendszer célja és funkciói

A **Studify** rendszere egy oktatási platform, amely lehetővé teszi kurzusok létrehozását, diákok és tanárok kezelését, posztok és beadandók megosztását, valamint az azonnali kommunikációt az oktatási közösségen belül. A platform célja, hogy átlátható, interaktív és valós idejű tanulási környezetet biztosítson.

1.3 Architektúra áttekintése

Az alkalmazás három fő részre tagolódik, amelyek együtt biztosítják a rendszer működését:

- **Felhasználói felület (Frontend)** a felhasználóval való közvetlen kapcsolatért felel, megjeleníti az adatokat és továbbítja a kéréseket a backend felé.
- **Backend** az rendszerlogikát valósítja meg, feldolgozza a kéréseket, kezeli a jogosultságokat és kapcsolatot tart az adatbázissal.
- **Adatbázis** a rendszer központi adattárolója, amely biztosítja az adatok strukturált tárolását, lekérdezését és integritását.

2. Felhasználói Felület

2.1 Technológiai alap

A felhasználói felület **Next.js** és **React.js** technológiákra épül, amelyek kombinálják a kliens- és szerveroldali renderelést (SSR/SSG). Ez lehetővé teszi a gyors oldalbetöltést, a dinamikus adatkezelést és a keresőoptimalizált tartalomkiszolgálást.

2.2 Felépítés és komponenslogika

A React komponensek moduláris felépítésben működnek. Az alkalmazás oldalszerkezete (layout) Next.js routingra épül, így minden fő funkció külön oldalon vagy szekcióban érhető el.

A szekciók lehetnek:

Fő modul	Route példa(i)	Leírás / képernyő funkció
Auth / User	/login, /register, /profile/:userId	Bejelentkezés, regisztráció, felhasználói profil
Courses	/courses, /courses/:courseId, /courses/:courseId/settings	Kurzuslista, kurzus főoldal, kurzusbeállítások
Members	/courses/:courseId/members	Kurzus tagjainak listája, szerepkörök
Posts	/courses/:courseId/posts, /posts/:postId, /posts/:postId/edit	Posztok listája, poszt részletei, poszt szerkesztése
Comments	/posts/:postId/comments	Kommentek megjelenítése és kezelése
Submissions	/posts/:postId/submissions, /submissions/:submissionId	Beadások listája és egy beadás részletei
Messaging	/posts/:postId/messages	Poszthoz kötött privát üzenetek
Notifications	/notifications	Értesítések listája, olvasás kezelése

2.3 UI komponensrendszer

A felhasználói élményt és vizuális konzisztenciát a **shadcn/ui** könyvtár biztosítja, amely **Tailwind CSS** alapú, könnyen testreszabható komponenseket kínál.

Az ikonrendszert a **Lucide Icons** szolgáltatja, amely modern, vektoros és egységes megjelenést biztosít a teljes alkalmazásban.

2.4 Backenddel való kommunikáció

Az alkalmazás a backenddel **REST API**-n keresztül kommunikál, míg a valós idejű eseményeket – például új üzenetek, kurzusbejegyzések, kommentek – **WebSocket** kapcsolaton keresztül fogadja.

3. Backend

A rendszer backendje réteges architektúrára épül, amely elkülöníti egymástól a prezentációs, oktatási és adatelérési logikát.

Az alkalmazás fő komponensei a **controllerek**, **servicek**, **repository**-k, az **adatbázis** és a valós idejű kommunikációt biztosító **WebSocket gateway**.

A backend felelős az oktatási folyamatok kezeléséért, az adatintegritás fenntartásáért, valamint a kliens- és szerveroldali kommunikáció biztonságos lebonyolításáért.

3.1 Controllerek

A controller réteg a kliensoldali kérések belépési pontja. A controllerek fogadják a **REST API**-hívásokat, elvégzik az alapvető **validálásokat**, és **továbbítják** az adatokat a megfelelő **service réteg** felé.

Minden fő funkcionális terület (pl. hitelesítés, kurzuskezelés, posztok, üzenetek) saját controllert kap, így az alkalmazás modulárisan bővíthető és karbantartható marad.

3.2 Servicek

A service réteg tartalmazza az alkalmazáslogikát. A controllerektől érkező adatokat feldolgozza, ellenőrzéseket végez, majd meghívja a repository réteget az adatbázisműveletekhez. A servicek biztosítják a adatkezelési folyamatként, az adatkonzisztenciát, valamint az események (pl. értesítések, WebSocket üzenetek) kiváltását.

3.3 Repository

A repository réteg felelős az adatbázissal való közvetlen kommunikációért. Az adatokat objektum-orientált módon kezeli a **Dizzle ORM** segítségével, így biztosítva a lekérdezések és módosítások típusbiztonságát és átláthatóságát. Ez a réteg elrejti az adatbázis logikai részleteit a servicek elől, megvalósítva az elválasztott felelősség elvét (Separation of Concerns).

3.4 Adatbázis

A backend egy relációs adatbázist használ, amely az entitások és kapcsolatok struktúráját szigorúan definiálja.

Az adatmodell külön dokumentumban kerül részletes bemutatásra, jelen fejezet csak az alkalmazás logikai integrációját érinti.

Az adatbázisban tárolt adatokhoz a repository réteg kizárólagos hozzáféréssel rendelkezik.

3.5 Websocket Gateway

A valós idejű eseménykezelést a WebSocket gateway biztosítja, amely folyamatos kapcsolatot tart fenn a kliensalkalmazásokkal. Az események – például új üzenetek, kurzusfrissítések vagy kommentek – a backend eseményrendszeréből indulnak, és a gateway-en keresztül azonnal

továbbításra kerülnek a kapcsolódott kliensek felé. Ez a megoldás biztosítja, hogy a felhasználók az alkalmazásban valós időben értesüljenek a változásokról.

4. Regisztráció folyamata (Példa az algoritmusról)

A regisztráció folyamata példaként mutatja be, miként halad át egy felhasználói művelet a rendszer rétegein:

a felületről (**Next.js alapú kliens**) a **controllereken** és **service** rétegen keresztül egészen az **adatbázisba történő mentésig**, valamint hogyan kapcsolódik ehhez a **valós idejű eseménykezelés**.

4.1 Felület (Frontend)

A felhasználó a regisztrációs űrlapon megadja a következő adatokat: név, e-mail cím, jelszó és opcionálisan profilkép.

Az űrlap egy **Next.js/React** komponensből áll, amely az adatokat lokálisan validálja (pl. kötelező mezők, jelszó erősség).

A beküldés (`onSubmit`) eseménykor a kliens egy **HTTP POST** kérést küld a backend felé az alábbi formátumban:

```
POST /api/auth/register
{
  "name": "Fülöp Miklós János",
  "email": "fulop.miklos.janos@diak.szbi-pg.hu",
  "password": "Password123!",
  "profile_picture": "profile1.png"
}
```

4.2 API és Controller réteg

A beérkező kérést az `AuthController` kezeli. Feladata a bemeneti adatok ellenőrzése, a hibák kezelése és a folyamat továbbítása a UserService-hez.

Példafolyamat:

1. A controller fogadja a kérést az `/auth/register` útvonalon.
2. A beérkező adatokat **DTO** (Data Transfer Object) segítségével validálja.
3. Amennyiben az adatok megfelelnek, meghívja a `UserService.registerUser()` metódust.
4. Hiba esetén (pl. már létező e-mail cím) HTTP hibaüzenetet küld vissza a kliensnek (`409 Conflict`).

A controller tehát a rendszer „bejárati kapuja”, amely nem tartalmaz rendszerlogikát, csak a kérések irányításáért felel.

4.3 Service réteg

A `UserService` végzi a tényleges regisztráció logikáját.

Fő lépései:

1. Ellenőrzi, hogy az e-mail cím nem szerepel-e már a `users` táblában.
2. Amennyiben új felhasználóról van szó, a jelszót **bcrypt** segítségével titkosítja.
3. Létrehozza az új felhasználói rekordot a `UserRepository` segítségével.
4. Generál egy **JWT tokent**, amely tartalmazza a felhasználó azonosítóját.
5. Kivált egy eseményt a **WebSocket gateway** felé (pl. `user_registered`), hogy a felhasználói lista azonnal frissülhessen más felhasználóknál (pl. admin nézetben).

A service tehát az alkalmazáslogika központi eleme, amely a validáció, adatkezelési folyamatként és eseményindítás feladatait látja el.

4.4 Repository réteg

A repository réteg az adatbázis-műveletekért felelős.

A `UserRepository` a **Drizzle ORM** segítségével kommunikál a `users` táblával.

A `create()` függvény elkészíti a megfelelő SQL-parancsot, és visszaadja a létrehozott entitást a service rétegnek.

```
async createUser(userData: NewUser) {
  const [insertedUser] = await db
    .insert(users)
    .values({
      name: userData.name,
      email: userData.email,
      password: userData.password,
      profile_picture: userData.profile_picture ?? null,
    })
    .returning();

  return insertedUser;
}
```

Az adat mentése után az új felhasználó adatai azonnal elérhetők a rendszer többi komponense számára.

4.5 Adatbázis réteg (mentési folyamat)

A regisztráció során az új felhasználó adatai a `users` táblába kerülnek mentésre. A mezők közül a `createdAt` értéke automatikusan generálódik az adatbázisban. A jelszó már **titkosított formában** kerül eltárolásra (bcrypt hash).

4.6 Websocket Gateway

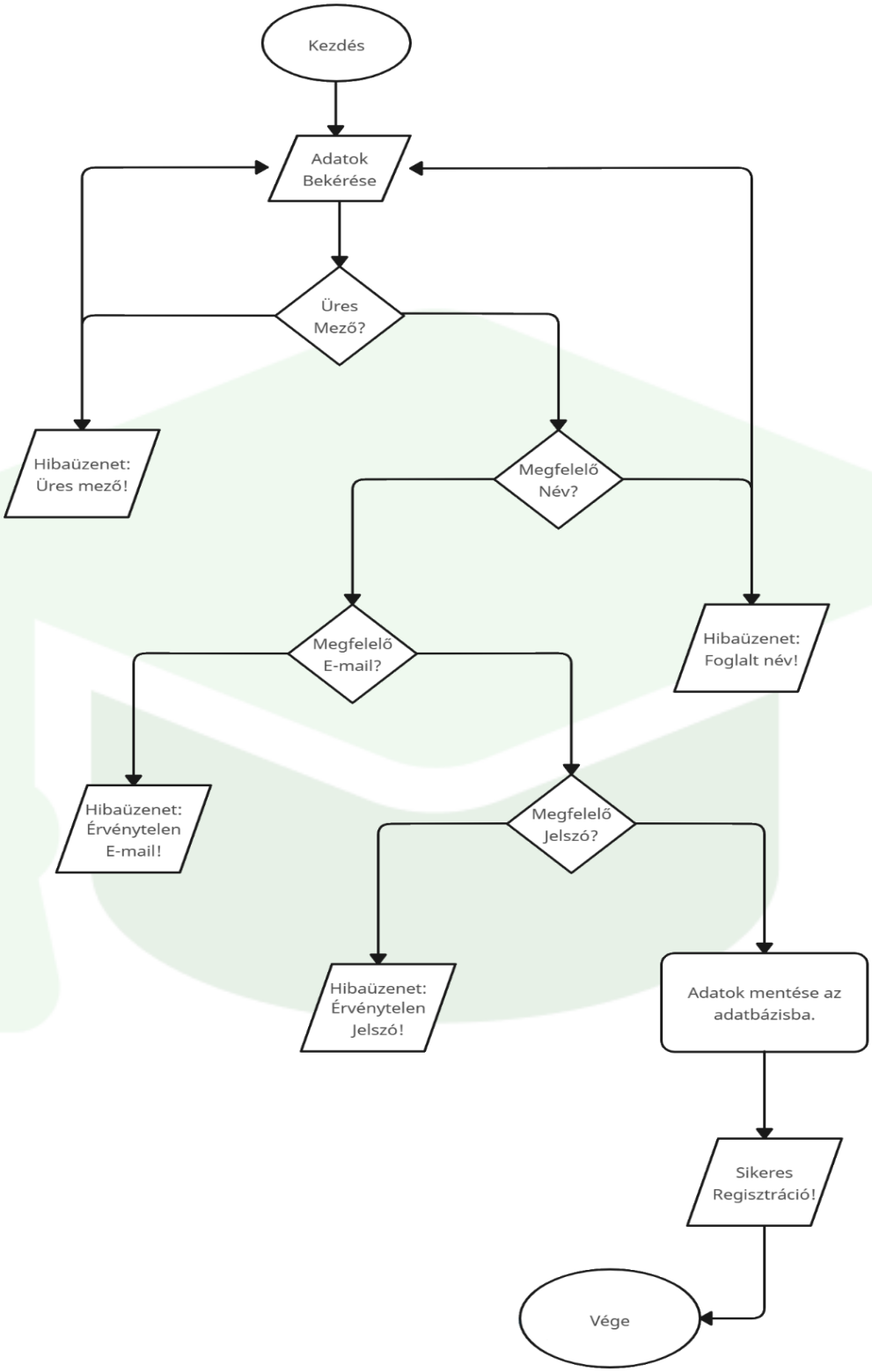
A sikeres regisztráció után a backend egy eseményt küld a **WebSocket gateway** felé, amely továbbítja azt az összes kapcsolódott kliensnek.

Az esemény neve: `user_registered`.

Az esemény tartalma:

```
{
  "event": "user_registered",
  "data": {
    "id": 42,
    "name": "Fülöp Miklós János",
    "email": "fulop.miklos.janos@diak.szbi-pg.hu"
  }
}
```

Azok a felhasználók, akik valós időben figyelik a felhasználói listát (pl. admin vagy tanári felület), azonnal látják az új felhasználó megjelenését, oldalfrissítés nélkül.

Ez a megoldás biztosítja a **valós idejű** szinkronizációt a rendszer komponensei között.

4.7 Folyamatábra